

# LINQ Select Operator/Method in C# with Examples

In this article, I will discuss the **LINQ Select Projection Operator/Method in C#** with Examples. Please read our previous article, which discusses the basic concepts of [LINQ Operators](#) in C#. At the end of this article, you will understand the following pointers related to LINQ Select Projection Operator in C#.

1. [What is Projection?](#)
2. [What are Projection Operators and Methods Available in LINQ?](#)
3. [How do you use the LINQ Select Method or Select Operator?](#)
4. [Examples to Understand Basic Projection of Data to the Same Type.](#)
5. [How do you project data to different Classes and Anonymous Types?](#)
6. [Performing Calculations on the Selected data using LINQ Select Operator.](#)
7. [How to Select Data with Index Value?](#)
8. [When should you use the LINQ Select Method in C#?](#)

**Note:** We will discuss each example using LINQ Query and Method Syntax. Again, when I am saying LINQ Operator or Method, the meaning is going to be same.

## What is Projection in LINQ?

Projection in LINQ is nothing but a mechanism used to select the data from a data source. You can select the data in the same form (i.e., the original form). It is also possible to create a new form of data by performing some operations on it. So, in simple words, we can say that Projection is an operation that converts an object into a new form that holds only those properties as per our requirements.

## What are Projection Methods or Operators Available in LINQ?

There are two methods available in the projection operator category in LINQ. They are as follows.

1. **Select**
2. **SelectMany**

In this article, we will discuss the **LINQ Select Method** with Examples, and in the next article, we will discuss the [LINQ SelectMany Method](#) with Examples.

## LINQ Select Operator/Method using C#:

The LINQ Select Projection Operator or Method can be used to format the query's result as per our requirement. The LINQ Select Operator can return a scalar value, a custom class, a collection of custom classes, or an anonymous type, which includes properties per our business requirements.

The Select Clause in SQL allows us to specify what columns we want to retrieve. Whether we want to retrieve all the columns or some of the columns that we need to specify in the select clause of the SQL Statement. In the same way, the LINQ Select operator allows us to specify what properties we want to retrieve. We need to specify whether we want to retrieve all or some of the properties in the Select Operator. The LINQ Select Method also allows us to perform some calculations.

## Examples to Understand LINQ Select Operator using C#:

Let us understand the LINQ Select Projection Operator with Examples using C# Language. Here, we are going to use a Console Application. So first, create a console application named **LINQDemo** (you can give any meaningful name). Then, add a new class file with the name **Employee.cs**. Once you add the **Employee.cs** class file, copy and paste the following code.

```
using System.Collections.Generic;
namespace LINQDemo
{
    public class Employee
    {
        public int ID { get; set; }
        public string FirstName { get; set; }
        public string LastName { get; set; }
        public int Salary { get; set; }
        public static List<Employee> GetEmployees()
        {
            List<Employee> employees = new List<Employee>
            {
                new Employee {ID = 101, FirstName = "Preety", LastName = "Tiwa",
Salary = 60000 },
                new Employee {ID = 102, FirstName = "Priyanka", LastName = "De",
Salary = 70000 },
                new Employee {ID = 103, FirstName = "Hina", LastName = "Sharma",
Salary = 80000 },
                new Employee {ID = 104, FirstName = "Anurag", LastName = "Moh",
Salary = 90000 },
                new Employee {ID = 105, FirstName = "Sambit", LastName = "Sata",
Salary = 100000 },
                new Employee {ID = 106, FirstName = "Sushanta", LastName = "Je",
Salary = 160000 }
            };
            return employees;
        }
    }
}
```

As you can see, we created the Employee class with the four properties: **ID**, **FirstName**, **LastName**, and **Salary**. We also created one static method to return the hard-coded list of employees, which will act as our data source. Let us discuss some examples to understand the LINQ Select Operator. The point that you need to remember is that while using the Query Syntax, I will use the term Select as Operator, and while using the Method Syntax, I will use the term Select as Method.

## Example to Understand LINQ Select Projection Operator/Method in C#:

Select all the Employee data from the data source using both the LINQ Method and Query Syntax. Please look at the following image, which shows Query and Method Syntax to fetch all the Employees. The following image is self-explained. You need to remember that it is not executed when we form the query. When we call the `ToList()` Method, `Sum()` Method, etc., or use the Query variable within a for-each loop, only the Query will be executed.

### Query Syntax:

```
IEnumerable<Employee> basicQuery = (from emp in Employee.GetEmployees()  
                                     select emp);
```

At this point the query is just created not executed. In order to execute the query we need to applied some methods to the query such as ToList() method or foreach loop. If you are not applying the ToList() method on the query then the data type is going to be IEnumerable<Employee>

```
List<Employee> basicQuery = (from emp in Employee.GetEmployees()  
                             select emp) ← Query  
                             .ToList(); ← Execution
```

At this time the query is created as well as executed as we applied the ToList() method on the query.

### Method Syntax:

```
IEnumerable<Employee> basicMethod = Employee.GetEmployees().ToList();
```

Modify the Main Method of the Program class as follows. In the below code, we are using the Select Method and Select Operator to return the data in its original shape. That means the return data shape will be identical to the Student class shape (i.e., the same properties).

```
using System;  
using System.Collections.Generic;  
using System.Linq;  
namespace LINQDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            //Using Query Syntax  
            List<Employee> basicQuery = (from emp in Employee.GetEmployees()  
                                         select emp).ToList();  
            foreach (Employee emp in basicQuery)  
            {  
                Console.WriteLine($"ID : {emp.ID} Name : {emp.FirstName}  
{emp.LastName}");  
            }  
            //Using Method Syntax  
            IEnumerable<Employee> basicMethod = Employee.GetEmployees().ToList  
            foreach (Employee emp in basicMethod)  
            {
```



```

        .ToList(); //At this Point the Query is Executed
foreach (var id in basicPropQuery)
{
    Console.WriteLine($"ID : {id}");
}
//Using Method Syntax
IEnumerable<int> basicPropMethod = Employee.GetEmployees()
    .Select(emp => emp.ID);
//At this Point the Query is Just Generated, Not Executed
foreach (var id in basicPropMethod) //At this Point the Query is {
    {
        Console.WriteLine($"ID : {id}");
    }
    Console.ReadKey();
}
}
}

```

## How Does It Work Internally?

- **Iteration:** The Select operator iterates over each element in the source sequence.
- **Transformation:** For each element, it applies the transformation logic defined in the lambda expression.
- **Result:** It produces a new sequence where each element is the result of the applied transformation on the corresponding element from the source sequence.

**Deferred Execution:** An important aspect of the Select operator in LINQ is that it uses deferred execution. This means that the actual transformation of elements doesn't happen when you define the Select call but when you iterate over the resulting sequence. This can be important for performance, especially with large data sets or complex queries.

## How do you Select a Few Properties using LINQ Select Operator or Select Method in C#?

We have discussed selecting all the properties and a single property using LINQ Select Projection Operator and Method. Now, let us proceed and try to understand how to select a few properties using LINQ Select Projection Operator and Method with an example.

Our requirement is to select only the Employee's First Name, Last Name, and Salary properties. We don't want to select the employee's ID property. For a better understanding, please have a look at the following image. In the code below, we select the First Name, Last Name, and Salary properties of the same Employee class. Later, I will show you how to project these properties to a different class and an anonymous type. With the Select Operator or Method, we create an instance of the Employee class and populate the First Name, Last Name, and Salary properties from the data source which we can access using the emp object.

//Query Syntax

```
IEnumerable<Employee> selectQuery = (from emp in Employee.GetEmployees()
```

Here we are selcting the First Name, Last Name and Salary properties to the same Employee class by excluding the ID property using Query Syntax

```
select new Employee()  
{  
    FirstName = emp.FirstName,  
    LastName = emp.LastName,  
    Salary = emp.Salary  
});
```

//Method Syntax

```
List<Employee> selectMethod = Employee.GetEmployees().
```

Here we are selcting the First Name, Last Name and Salary properties to the same Employee class by excluding the ID property using Method Syntax

```
Select(emp => new Employee()  
{  
    FirstName = emp.FirstName,  
    LastName = emp.LastName,  
    Salary = emp.Salary  
}).ToList();
```

The Complete Example Code is Given Below.

```
using System;  
using System.Collections.Generic;  
using System.Linq;  
namespace LINQDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            //Query Syntax  
            IEnumerable<Employee> selectQuery = (from emp in Employee.GetEmployees()  
                                                select new Employee()  
                                                {  
                                                    FirstName = emp.FirstName,  
                                                    LastName = emp.LastName,  
                                                    Salary = emp.Salary  
                                                });  
  
            foreach (var emp in selectQuery)  
            {  
                Console.WriteLine($" Name : {emp.FirstName} {emp.LastName} Sal  
{emp.Salary} ");  
            }  
  
            //Method Syntax  
            List<Employee> selectMethod = Employee.GetEmployees().  
                Select(emp => new Employee()  
                {
```

```

        FirstName = emp.FirstName,
        LastName = emp.LastName,
        Salary = emp.Salary
    }).ToList();
    foreach (var emp in selectMethod)
    {
        Console.WriteLine($" Name : {emp.FirstName} {emp.LastName} Sal
{emp.Salary} ");
    }
    Console.ReadKey();
}
}
}

```

## How do you Select a Few Properties to a Different class using a LINQ Select Operator?

It is also possible to project or select the data to a different class using the LINQ Select Operator or Method. In our previous example, we have seen how to select a few properties (First Name, Last Name, and Salary properties) to the same class using the LINQ Select Projection Operator. Let us create a new class with the above three properties, and we will project the data to this class. So, please create a new class file named **EmployeeBasicInfo.cs** and copy and paste the following code.

```

namespace LINQDemo
{
    public class EmployeeBasicInfo
    {
        public string FirstName { get; set; }
        public string LastName { get; set; }
        public int Salary { get; set; }
    }
}

```

We need to select the First Name, Last Name, and Salary properties to the above newly created EmployeeBasicInfo class. For a better understanding, please have a look at the below image. Now, with the Select Method or Operator, we are not creating an instance of the Employee class. We are creating an instance of the EmployeeBasicInfo class and populating the FirstName, LastName, and Salary properties from the data source we can access using the emp object.

//Query Syntax

```
IEnumerable<EmployeeBasicInfo> selectQuery = (from emp in Employee.GetEmployees()  
                                              select new EmployeeBasicInfo()  
                                              {  
                                                  FirstName = emp.FirstName,  
                                                  LastName = emp.LastName,  
                                                  Salary = emp.Salary  
                                              });
```

The only difference is that instead of using Employee type, now we are using EmployeeBasicInfo type

//Method Syntax

```
List<EmployeeBasicInfo> selectMethod = Employee.GetEmployees().  
    Select(emp => new EmployeeBasicInfo()  
    {  
        FirstName = emp.FirstName,  
        LastName = emp.LastName,  
        Salary = emp.Salary  
    }).ToList();
```

The Complete Example Code is Given Below.

```
using System;  
using System.Collections.Generic;  
using System.Linq;  
namespace LINQDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            //Query Syntax  
            IEnumerable<EmployeeBasicInfo> selectQuery = (from emp in  
Employee.GetEmployees()  
                                                         select new EmployeeBasicInfo()  
                                                         {  
                                                             FirstName = emp.FirstName,  
                                                             LastName = emp.LastName,  
                                                             Salary = emp.Salary  
                                                         });  
  
            foreach (var emp in selectQuery)  
            {  
                Console.WriteLine($" Name : {emp.FirstName} {emp.LastName} Sal  
{emp.Salary} ");  
            }  
            //Method Syntax  
            List<EmployeeBasicInfo> selectMethod = Employee.GetEmployees().  
                Select(emp => new EmployeeBasicInfo()  
                {  
                    FirstName = emp.FirstName,
```

```

        LastName = emp.LastName,
        Salary = emp.Salary
    }).ToList();

    foreach (var emp in selectMethod)
    {
        Console.WriteLine($" Name : {emp.FirstName} {emp.LastName} Sal
{emp.Salary} ");
    }
    Console.ReadKey();
}
}
}

```

## How do you project the data to Anonymous Type using LINQ Select Operator/Method?

Instead of projecting the data to a particular type like Employee or EmployeeBasicInfo, we can also project the data to an anonymous type in LINQ using the Select Method or Operator. For a better understanding, please have a look at the below image. Here, we are creating an anonymous object (i.e., creating an object without specifying the type) and creating and populating the FirstName, LastName, and Salary properties from the data source which we can access using the emp object.

```

//Query Syntax
var selectQuery = (from emp in Employee.GetEmployees()
                    select new
                    {
                        FirstName = emp.FirstName,
                        LastName = emp.LastName,
                        Salary = emp.Salary
                    });

//Method Syntax
var selectMethod = Employee.GetEmployees().
    Select(emp => new
    {
        FirstName = emp.FirstName,
        LastName = emp.LastName,
        Salary = emp.Salary
    }).ToList();

```

The Complete Example Code is Given Below.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{

```

```

class Program
{
    static void Main(string[] args)
    {
        //Query Syntax
        var selectQuery = (from emp in Employee.GetEmployees()
                           select new
                           {
                               FirstName = emp.FirstName,
                               LastName = emp.LastName,
                               Salary = emp.Salary
                           });

        foreach (var emp in selectQuery)
        {
            Console.WriteLine($" Name : {emp.FirstName} {emp.LastName} Salary : {emp.Salary} ");
        }

        //Method Syntax
        var selectMethod = Employee.GetEmployees().
            Select(emp => new
            {
                FirstName = emp.FirstName,
                LastName = emp.LastName,
                Salary = emp.Salary
            }).ToList();

        foreach (var emp in selectMethod)
        {
            Console.WriteLine($" Name : {emp.FirstName} {emp.LastName} Salary : {emp.Salary} ");
        }
        Console.ReadKey();
    }
}

```

## How do you Perform Calculations on Selected Data using the LINQ Select Operator?

Let me first explain what we want to achieve. We want to perform the following calculations on the selected employee data. We need to calculate the Annual Salary and merge the First and Last Name as Full Name in the output.

1. **AnnualSalary = Salary\*12**
2. **FullName = FirstName + " " + LastName**

Once we do the above calculation, we need to project the ID, AnnualSalary, and FullName to an anonymous type using the LINQ Projection Operator. For a better understanding, please have a look at the following image.

```
//Query Syntax
var selectQuery = (from emp in Employee.GetEmployees()
    select new
    {
        EmployeeId = emp.ID,
        FullName = emp.FirstName + " " + emp.LastName,
        AnnualSalary = emp.Salary * 12
    });

//Method Syntax
var selectMethod = Employee.GetEmployees().
    Select(emp => new
    {
        EmployeeId = emp.ID,
        FullName = emp.FirstName + " " + emp.LastName,
        AnnualSalary = emp.Salary * 12
    }).ToList();
```

**The Complete Example Code is Given Below.**

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Query Syntax
            var selectQuery = (from emp in Employee.GetEmployees()
                               select new
                               {
                                   EmployeeId = emp.ID,
                                   FullName = emp.FirstName + " " + emp.LastName,
                                   AnnualSalary = emp.Salary * 12
                               });

            foreach (var emp in selectQuery)
            {
                Console.WriteLine($" ID {emp.EmployeeId} Name : {emp.FullName}
Salary : {emp.AnnualSalary} ");
            }
            //Method Syntax
            var selectMethod = Employee.GetEmployees().
                Select(emp => new
                {
                    EmployeeId = emp.ID,
```

```

        FullName = emp.FirstName + " " +
emp.LastName,

        AnnualSalary = emp.Salary * 12
    }).ToList();
    foreach (var emp in selectMethod)
    {
        Console.WriteLine($" ID {emp.EmployeeId} Name : {emp.FullName}
Salary : {emp.AnnualSalary} ");
    }
    Console.ReadKey();
}
}
}

```

## How do you Select Data with Index Value using LINQ Select Projection Operator?

It is also possible to select values using an integral index. The index is 0 based. The index will be from 0 to 4 if the query fetches five records. It's a unique value to each record we select or project from the data source.

```

//Query Syntax
var query = (from emp in Employee.GetEmployees().Select((value, index) => new { value, index })
select new
{
    IndexPosition = emp.index,
    FullName = emp.value.FirstName + " " + emp.value.LastName,
    Salary = emp.value.Salary
}).ToList();

//Method Syntax
var selectMethod = Employee.GetEmployees().
    Select((emp, index) => new
    {
        IndexPosition = index,
        FullName = emp.FirstName + " " + emp.LastName,
        Salary = emp.Salary
    });

```

The Complete Example Code is Given Below.

```

using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Query Syntax
            var query = (from emp in Employee.GetEmployees().Select((value, index) => new
{ value, index })

```

```

        select new
        {
            //Index is 0-Based, and always increases by 1
            IndexPosition = emp.index,
            FullName = emp.value.FirstName + " " + emp.value.
            emp.value.Salary
        }).ToList();
    foreach (var emp in query)
    {
        Console.WriteLine($" Position {emp.IndexPosition} Name : {emp.
Salary : {emp.Salary} ");
    }
    //Method Syntax
    //Projects each element of a sequence into a new form by incorpora
element's index.
    var selectMethod = Employee.GetEmployees().
        Select((emp, index) => new
        {
            //Index is 0-Based, and always i
1
            IndexPosition = index,
            FullName = emp.FirstName + " " +
emp.LastName,
            emp.Salary
        });
    foreach (var emp in selectMethod)
    {
        Console.WriteLine($" Position {emp.IndexPosition} Name : {emp.
Salary : {emp.Salary} ");
    }
    Console.ReadKey();
}
}
}

```

## When should you use the LINQ Select Operator in C#?

The Select operator projects each sequence element into a new form. This is analogous to the SELECT statement in SQL, which picks out specific columns of data from a table. Here are several scenarios when you might use it:

- **Transforming Data Types or Shapes:** When you need to convert elements of a collection from one type to another or when you need to transform them into a new form. For instance, extracting specific properties from objects to create a new collection of simpler objects or anonymous types.
- **Data Projection:** When you're querying a data source, like a database or an XML file, and you want to project the data into a different form. This is particularly common in database operations where you only need specific columns from a table.
- **Applying Calculations or Methods:** If you need to apply a calculation or method to each element in a collection, Select can be used to produce a new collection with the results. For example, calculating the square of each number in a list.

- **Fluent Method Chaining:** LINQ Select is often used in conjunction with other LINQ methods like Where, OrderBy, GroupBy, etc., in a fluent syntax to perform complex queries and transformations in a readable and concise manner.
- **Performance Optimizations:** In some cases, using Select can be more efficient, especially when working with large collections or databases, as it allows for the transformation of data as it is being retrieved or processed.
- **Index-based Selection:** The Select operator can also provide the index of each element in the lambda expression, which can be used for more complex transformations that might depend on the element's position within the collection.
- **Creating Computed Values:** When you want to compute and add new values to each element in a collection based on some criteria or calculations. This helps create derived values or aggregations.
- **Querying and Filtering:** When you want to filter or query data based on certain conditions, create a new collection of matching elements. Allows you to refine your data set based on specific criteria.

That's it for today. In the next article, I will discuss the [SelectMany Projection Operator in C#](#) with Examples. In this article, I try to explain the LINQ Select Projection Operator/Method in C# with examples. I hope you enjoy this LINQ Select Projection Operator/Method in C# with Examples article.